



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)
КАФЕДРА «Информационная безопасность» (ИУ8)

Лабораторная работа № 2
ПО КУРСУ
«Теория игр в системах защиты информации»
на тему «Решение матричных игр с нулевой суммой
аналитическим (матричным) и числовым (Брауна-
Робинсона) методами»

Студент

ИУ8-22М
(Группа)

Е. В. Умеренко
(И. О. Фамилия)

Преподаватель:

П. В. Захаров
(И.О. Фамилия)

Цель работы

Изучить аналитический (метод обратной матрицы) и численный (метод Брауна-Робинсона) подходы [1] к нахождению смешанных стратегий в антагонистической игре двух лиц в нормальной форме.

Постановка задачи

Решите приведенную в варианте задания игру (найдите цену игры и оптимальные стратегии обоих игроков) методами обратной матрицы и Брауна-Робинсона. Сравните полученные результаты.

Ход работы

Пусть (3×3) – игра Γ задана матрицей:

$$C = \begin{pmatrix} 9 & 10 & 13 \\ 1 & 18 & 11 \\ 17 & 4 & 0 \end{pmatrix}$$

Аналитический метод.

Теорема 1. Вполне смешанная игра $(m \times n)$ – игра Γ имеет единственную ситуацию равновесия (x^*, y^*) и квадратичную матрицу $(m = n)$;

если цена игры $v \neq 0$, то матрица C невырожденная и

$$x^* = \frac{C^{-1}u^T}{u C^{-1}u^T}, \quad y^* = \frac{uC^{-1}}{u C^{-1}u^T}, \quad v = \frac{1}{u C^{-1}u^T} \quad (1)$$

Расчёт по формулам (1) дает следующее аналитическое решение задачи, рисунок 1:

$$x^* = (0.1739, 0.6739, 0.1522)$$

$$y^* = (0.1087, 0.6739, 0.2174)$$

$$v = 13.869565$$

АНАЛИТИЧЕСКИЙ МЕТОД

Цена игры: 13.8696

Стратегия А: [np.float64(0.1087), np.float64(0.6739), np.float64(0.2174)]

Стратегия В: [np.float64(0.1739), np.float64(0.6739), np.float64(0.1522)]

Рисунок 1. Результат расчета аналитическим методом на Python

Реализуем теперь алгоритм Брауна-Робинсона. Пусть на первом шаге игроки выбрали стратегии x_1, y_1 . Учитывая, что игрок А выбрал x_1 , игрок В мог получить один из выигрышей (6, 18, 5). А если игрок В выбрал y_1 , то возможные выигрыши игрока А были (6, 17, 9).

Результаты методом Брауна-Робинсона были получены через 3630 итераций, рис.2.

№	А	В	Выигрыш А (x1,x2,x3)	Проигрыш В (y1,y2,y3)	v_up	v_low
1	x1	y1	6, 17, 9	6, 18, 5	17.0000	5.0000
2	x2	y3	11, 32, 28	23, 31, 20	16.0000	10.0000
3	x2	y3	16, 47, 47	40, 44, 35	15.6667	11.6667
4	x2	y3	21, 62, 66	57, 57, 50	16.5000	12.5000
5	x3	y3	26, 77, 85	66, 70, 69	17.0000	13.2000
6	x3	y1	32, 94, 94	75, 83, 88	15.6667	12.5000
7	x2	y1	38, 111, 103	92, 96, 103	15.8571	13.1429
8	x2	y1	44, 128, 112	109, 109, 118	16.0000	13.6250
9	x2	y1	50, 145, 121	126, 122, 133	16.1111	13.5556
10	x2	y2	68, 158, 134	143, 135, 148	15.8000	13.5000
11	x2	y2	86, 171, 147	160, 148, 163	15.5455	13.4545
12	x2	y2	104, 184, 160	177, 161, 178	15.3333	13.4167
13	x2	y2	122, 197, 173	194, 174, 193	15.1538	13.3846
14	x2	y2	140, 210, 186	211, 187, 208	15.0000	13.3571
15	x2	y2	158, 223, 199	228, 200, 223	14.8667	13.3333
16	x2	y2	176, 236, 212	245, 213, 238	14.7500	13.3125
17	x2	y2	194, 249, 225	262, 226, 253	14.6471	13.2941
18	x2	y2	212, 262, 238	279, 239, 268	14.5556	13.2778
19	x2	y2	230, 275, 251	296, 252, 283	14.4737	13.2632
... (всего итераций: 3630, показаны первые 20)						

Рисунок 2. Результат расчета методом Брауна-Робинсона на Python

РЕЗУЛЬТАТ после 3630 итераций:

Цена игры (средняя): 13.8255

Стратегия А: [0.1551, 0.7355, 0.1094]

Стратегия В: [0.1116, 0.6738, 0.2146]

Метод Брауна-Робинсона применим для матричных игр, но требует большого числа итераций для достижения приемлемой точности. Аналитический метод универсален и предпочтителен для игр малой размерности.

Приложение А.

```
import numpy as np
import matplotlib.pyplot as plt

matrix = np.array([
    [6, 18, 5],
    [17, 13, 15],
    [9, 13, 19]
])

def analytical_method(C):
    C_inv = np.linalg.inv(C)
    total_sum = np.sum(C_inv)
    v = 1 / total_sum
    row_sums = np.sum(C_inv, axis=1)
    x = row_sums / total_sum
    col_sums = np.sum(C_inv, axis=0)
    y = col_sums / total_sum
    return v, x, y

def solve_brown_robinson(C, target_eps, max_iter=10000,
max_display_rows=20):

    m, n = C.shape
    acc_win_A = [0.0, 0.0, 0.0]
    acc_loss_B = [0.0, 0.0, 0.0]
    counts_A = [0, 0, 0]
    counts_B = [0, 0, 0]

    v_up_history = []
    v_low_history = []
    log_rows = []

    k = 1
    curr_A = 0
    curr_B = 0
    epsilon = 1.0

    header = f"{'N':<6} {'A':<6} {'B':<6} {'Выигрыш A (x1,x2,x3)':<25}
{'Проигрыш B (y1,y2,y3)':<25} {'v_up':<8} {'v_low':<8}"
```

```

log_rows.append(header)
log_rows.append("-" * 90)

while epsilon > target_eps and k <= max_iter:
    counts_A[curr_A] += 1
    counts_B[curr_B] += 1

    for i in range(m):
        acc_win_A[i] += C[i][curr_B]
    for j in range(n):
        acc_loss_B[j] += C[curr_A][j]

    v_up_k = max(acc_win_A) / k
    v_low_k = min(acc_loss_B) / k

    v_up_history.append(v_up_k)
    v_low_history.append(v_low_k)

    epsilon = v_up_k - v_low_k

    win_str = f"{int(acc_win_A[0])}, {int(acc_win_A[1])},
{int(acc_win_A[2])}"
    loss_str = f"{int(acc_loss_B[0])}, {int(acc_loss_B[1])},
{int(acc_loss_B[2])}"

    row = f"{k:<6} x{curr_A+1:<5} y{curr_B+1:<5} {win_str:<25}
{loss_str:<25} {v_up_k:<8.4f} {v_low_k:<8.4f}"
    log_rows.append(row)

    curr_A = acc_win_A.index(max(acc_win_A))
    curr_B = acc_loss_B.index(min(acc_loss_B))

    k += 1

steps = k - 1
x_strat = [round(c / steps, 4) for c in counts_A]
y_strat = [round(c / steps, 4) for c in counts_B]

v_res = (v_up_history[-1] + v_low_history[-1]) / 2

if steps > max_display_rows:
    display_log = log_rows[:max_display_rows + 1]
    display_log.append(f"... (всего итераций: {steps}, показаны первые
{max_display_rows})")
else:
    display_log = log_rows

    return v_res, x_strat, y_strat, v_up_history, v_low_history, steps,
display_log

```

```

def main():
    print("АНАЛИТИЧЕСКИЙ МЕТОД")
    v_analytical, x_analytical, y_analytical = analytical_method(matrix)
    print(f"Цена игры: {v_analytical:.4f}")
    print(f"Стратегия A: {[round(x,4) for x in x_analytical]}")
    print(f"Стратегия B: {[round(y,4) for y in y_analytical]}")

    print("\n" + "="*70)
    print("ЧИСЛЕННЫЙ МЕТОД БРАУНА-РОБИНСОНА")
    target_eps = 0.1
    print(f"Целевая погрешность:  $\varepsilon$  = {target_eps}\n")

    v_brown, x_brown, y_brown, v_up_hist, v_low_hist, iterations,
log_table = solve_brown_robinson(
    matrix, target_eps, max_display_rows=20
)

    print("\n".join(log_table))

    print(f"\nРЕЗУЛЬТАТ после {iterations} итераций:")
    print(f"Цена игры (средняя): {v_brown:.4f}")
    print(f"Стратегия A: {x_brown}")
    print(f"Стратегия B: {y_brown}")

    print("\n" + "-"*70)
    print("СРАВНЕНИЕ С АНАЛИТИЧЕСКИМ РЕШЕНИЕМ:")
    print(f"    Аналитическая цена игры: {v_analytical:.6f}")
    print(f"    Численная оценка:          {v_brown:.6f}")
    print(f"    Абсолютная погрешность:    {abs(v_analytical - v_brown):.6f}")
    print(f"    Количество итераций:        {iterations}")

    '''plt.figure(figsize=(10, 5))
    plt.plot(range(1, len(v_up_hist)+1), v_up_hist, label='v_up (верхняя
оценка)')
    plt.plot(range(1, len(v_low_hist)+1), v_low_hist, label='v_low (нижняя
оценка)')
    plt.axhline(y=v_analytical, color='r', linestyle='--',
label='Аналитическая цена игры')
    plt.xlabel('Номер итерации k')
    plt.ylabel('Оценка цены игры')
    plt.title('Сходимость метода Брауна-Робинсон')
    plt.legend()
    plt.grid(True)
    plt.show()'''

```